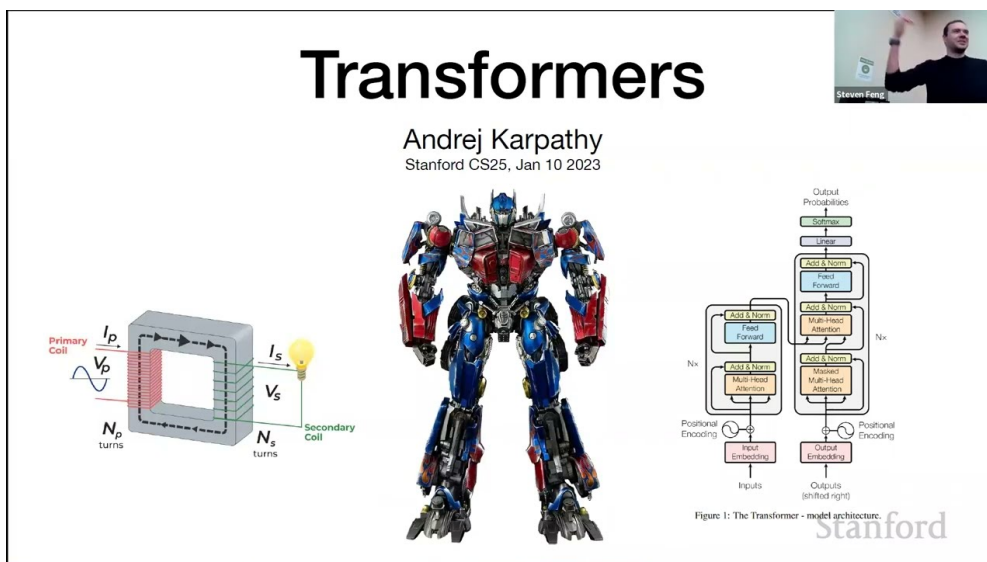


课程笔记

[CS25] Introduction to Transformers — Andrej Karpathy

基于公开课程资料整理

2023 年 2 月 23 日



视频作者/频道: Stanford CS25

发布日期: 2023 年 2 月 23 日

视频时长: 约 64 分钟

视频链接: <https://www.youtube.com/watch?v=XfpMkf4rD6E>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：课程定位与讲者承诺	3
1.1 Karpathy 的宣言	3
1.2 封面与滑动画面	3
1.3 课程结构与观众期待	3
1.4 本章小结	4
2 Transformer 架构解构	4
2.1 自注意力即图上的消息传递	4
2.2 Attention 头与可解释性	4
2.3 Transformers Block 与归一化	4
2.4 位置编码与尺度控制	5
2.5 本章小结	5
3 Scaling 与实作策略	6
3.1 从算法导向到 Compute/Data 导向	6
3.2 训练流水线：预训练 → 微调 → RLHF	6
3.3 Prompt 设计就是 “thick sequence” 的编排	6
3.4 RNN vs Transformer：串行 vs 并行	7
3.5 本章小结	7
4 应用与案例剖析	7
4.1 minGPT / nanoGPT 的教学案例	7
4.2 多模态与 RL 方向的示例	8
4.3 未来趋势与人机交互	8
4.4 Slides/Frames 与素材策略	8
4.5 本章小结	9
5 工程实践与复现	9
5.1 数据与计算预算	9
5.2 调试与观察	9
5.3 复现脚本示例	10
5.4 本章小结	10
6 Transformer 演化与 Karpathy 视角	10
6.1 关键变体对照	10
6.2 Karpathy 对稳定性的解读	11
6.3 本章小结	11
7 讲座时间线与主题剖析	12
7.1 拆解主要章节	12
7.2 Q&A 与反思	12
7.3 本章小结	12

8 实践问题清单	12
8.1 常见问题与应对	12
8.2 工具与自动化	13
8.3 本章小结	13
9 Prompt 协同与人类交互	13
9.1 Chain-of-thought 与 prompt 结构	13
9.2 Prompt 流程治理与 review	14
9.3 本章小结	14
10 评估与治理实务	14
10.1 评估矩阵与 failure signature	14
10.2 对齐治理的实战	15
10.3 本章小结	15
11 部署与可观察性	15
11.1 部署准备 checklist	15
11.2 可观察性与 drift detection	16
11.3 本章小结	16
12 复现与研究方向	16
12.1 数据与模型构建 pipeline	16
12.2 推理效率与对齐探索	17
12.3 本章小结	17
13 总结与延伸	18
13.1 拓展阅读	18

1 引言：课程定位与讲者承诺

1.1 Karpathy 的宣言

Andrej Karpathy 从 Tesla AI 团队转入 OpenAI 时，带着“Transformers United”的理念来到 CS25 的第十一讲。开场他强调，本课程不是讲变形金刚机器人，而是讲**能够从自然语言进化出来的深度学习架构**。基于这一背景，整堂课的节奏按照构建模块 → 规模 → 应用的链路展开，核心动力是让“Transformer 作为通用文本计算器”的能力变得可感知。

1.2 封面与滑动画面

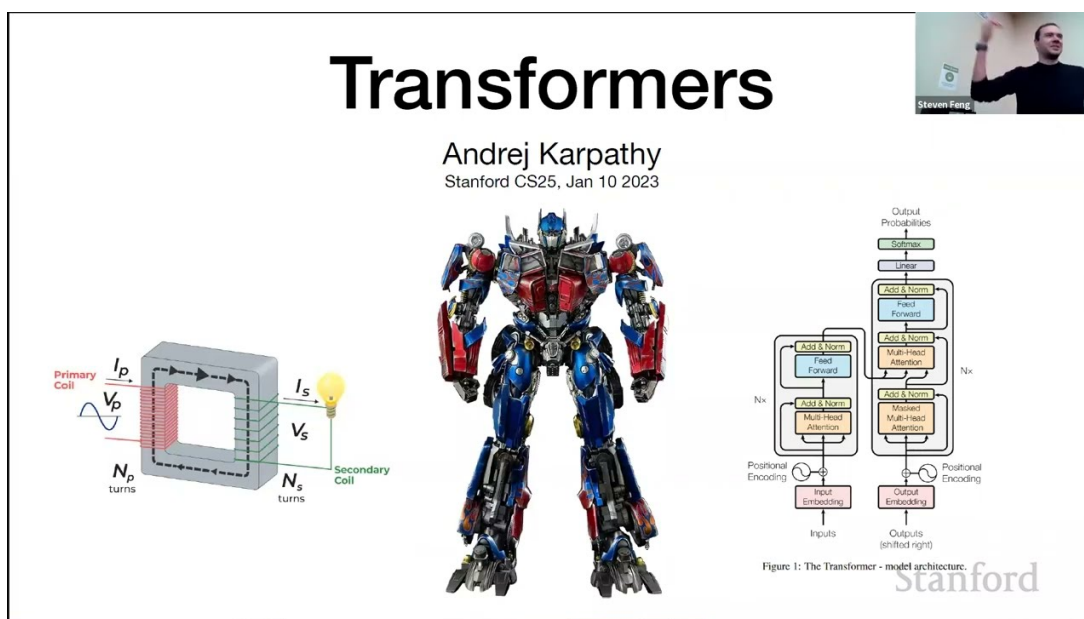


图 1: Lecture 11 的正式封面 / slide 画面。

教学目标与约束

Karpathy 设定的三条主线：理解 Transformer 的图模型本质、把握其训练稳定性的技巧、以及透过 scaling + prompt 设计让它成为“文本通用计算器”。这种教学逻辑帮助我们 from 高层抽象下来，再逐步展开放大细节。

1.3 课程结构与观众期待

CS25 的听众包含对 Transformer 有初步接触的研究生与希望快速掌握前沿架构的工程师。讲者借助引人注目的视觉（如开场幻灯片）、模拟对话（“this is not about robots”），以及随堂问答，用**讲义 + slides**的组合将复杂概念渐进式拆解。

⁰ 视频画面时间区间：00:00:05–00:00:20，以课程开场幻灯片为素材，遵守 slides/frames 引导将关键 slide 纳入 PDF。

观众的实时期待

- 初始阶段要清楚 Transformer 在 token 之间建立了怎样的动态图
- 中段期待看到 scaling 后的实证成果（从 GPT, Whisper 到 RLHF）
- 末段希望掌握能够复现的 minGPT/nanoGPT 示例，确认理论可落地

1.4 本章小结

本节强调 Karpathy 对建模、训练、部署三段流水线的承诺，并通过封面 slide 的视觉符号提前告诉读者：本笔记将同时包含讲义、关键截图与时间戳引用。

2 Transformer 架构解构

2.1 自注意力即图上的消息传递

Karpathy 将自注意力的操作描述为 “communication phase”，也就是 “*a data dependent message passing on directed graphs*”。序列中的每个 token 相当于图中的节点，注意力权重决定了边的强度，这个图本质上在前向传播时动态建立。

自注意力的图论视角

每个位置都维护一个向量（图节点），注意力机制在每层完成三件事：收集邻居表示（消息）、根据 query/k/v 重新计算边权（调度），然后用 softmax 门控并更新节点。Sub-token 之间的 bidirectional（编码器）或 causal（解码器）约束可以直接映射到图的连通模式。

2.2 Attention 头与可解释性

讲者进一步用 “attention head 就像不同角度观察图” 的比喻，说明 Transformer 通过多个 head 并行估算多种关系：有的 head 学习实体相似度、有的 head 捕捉位置偏移，而 residual + LayerNorm 则汇总这些视图。

Head specialization

- Head 1: 关注 token 之间的词汇相似性（例如名词与其修饰的形容词）
- Head 2: 担任 Position Bias（主动关注远距离 token）
- Head 3: 相当于 “copy” 操作，帮助生成时穿插上下文

2.3 Transformers Block 与归一化

一个典型的 Transformer block 包含两套 LayerNorm（Pre-Norm 架构）、多头自注意力、残差连接、以及 2 层 MLP。Karpathy 特别指出 “Transformer 是一个浅而宽的 graph，supervision 到 input 的 hop 数非常少”，因此 “*gradients flow very easily*”。

在 self-attention 中：

- Query 表示当前 token 的状态，其余 token 提供 Key 与 Value

- 通过 scaled dot-product 计算 attention score，经过 softmax 调味
- Multi-head 并行计算不同子空间的注意力，并在最后线性投影融合结果

组件	作用	数据流
LayerNorm	规范化输入，促使梯度更平滑	每次 residual 前后都进行 affine 归一化
Multi-head	并行估算多种关系	每个 head 生成 Q/K/V，softmax 后加权并拼接
MLP	提供非线性变换	2 层 Dense + GeLU，引入 cross-token coupling

表 1: Transformer block 中关键组件的角色与数据流。

Block 中的稳定配方

- Pre-Norm 让梯度从监督端迅速穿过 self-attn 与 MLP，而不用担心梯度消失
- 残差连接与多头 attention 形成的跳跃路径降低了局部最小值
- LayerNorm 控制所有激活尺度，避免因 scale 过大而发散

2.4 位置编码与尺度控制

位置编码负责补位自注意力固有的置换不变性。Karpathy 在讲解时提到，除了传统的正弦/可学习编码，旋转位置编码（RoPE）、ALiBi 等都在控制 “attention 中的加权 bias”，帮助模型在 **高频变化与长序列依赖**间取得平衡。无论何种位置编码，LayerNorm 依然是控制整体尺度的关键，其作用类似于插入一个 “scale locker”，保证 stacking 之后的残差路径不爆炸。

方案	描述	适用场景
绝对位置编码	正弦/余弦函数 + 可学习参数	经典 Transformer
RoPE	将位置嵌入旋转到 embedding 空间，使 attention score 成本中包含相对角度	长序列推理
ALiBi	在 attention score 上加线性 bias，无需显式位置 embedding	需要线性化推理时（如 GPT 系列）

表 2: 常见位置编码方案对比。

2.5 本章小结

自注意力就是图上的消息传递，Transformer block 正是将快路径与宽度结合，用 LayerNorm 保障稳定。理解这一结构是后续 scaling 与应用扩展的基础。

3 Scaling 与实作策略

3.1 从算法导向到 Compute/Data 导向

Karpathy 归功于他的同事们展示的实验: “if you scale a large neural network on large data set you can get very strong performance”, 这句话是迁移到 “scale first” 思维的分水岭。此后各类研究都证明: 在顺畅的训练管线下, 算法微调 (如 attention 变体) 对模型性能的提升远不及投入更多 compute 与更多 data。

在此过程中, 数据清理 (去重、规范化) 和 compute 节奏同样重要: Karpathy 提醒我们 “worry about compute and data” 意味着每笔 GPU 消耗都要有明确的目标, 避免在低质量数据上浪费训练预算。

Scaling Recipe

- 扩大 token 数量、模型参数规模与 batch size
- 放眼 compute/data 成本 (Karpathy 反复强调 “worry about compute and data”)
- 通过 prompt 设计或 RLHF 定制输出行为, 而不是为模型增加复杂模块

3.2 训练流水线: 预训练 → 微调 → RLHF

Karpathy 提到 ChatGPT 的成功并非一蹴而就, 而是三步走: 在大规模 WebText 上进行自回归预训练 → 在指令式数据上做微调 → 通过 RLHF 调整可控性。这条流水线把 “scale first” 的模型交出, 再在下游任务上用数据和人类反馈精细打磨。

阶段	输入	目标
预训练	Common Crawl, WebText	学习通用语言建模, 生成下一个 token
指令微调	Prompt-response pairs	让模型学会遵守格式与上下文
RLHF	人类偏好比较	微调 reward model + PPO, 使输出更自然

表 3: 训练流水线各阶段的输入与目标。

Pipeline 的可扩展性

这样的流水线允许我们把大模型当作 “基础模型”, 通过微调与 RLHF 只改训练后的少量参数 (或者通过调用 API) 即可跨任务复用, 是 ChatGPT、Claude 等产品背后的共通逻辑。

3.3 Prompt 设计就是 “thick sequence” 的编排

在讲述 prompt 时 Karpathy 说到: “you can design the sequence in the prompt ... because transformer is both powerful but also is trained on large enough very hard dataset, it kind of becomes this general purpose text computer”。提示工程最终是把 prompt 当作 “thick sequence” (一个紧密依赖上下文的程序), 提前设置好任务约束与示例。

他进一步指出, prompt 可以包含 chain-of-thought、具体角色设定或多阶段提示, 并通过 “few-shot” 方式把示例嵌入开头, 借助 attention 的长距离路径让模型直接沿着提示结构输出结果, 而无需改动参数。

Prompt 作为程序加载器

把 prompt 视为 “sequence of instructions”: 在前几个 token 中注入格式 (“输入-输出对”、“角色设定”), Transformer 会将这些 token 视为 part of the computation graph, 使后续 token 在同一个 “graph execution” 中完成任务。

3.4 RNN vs Transformer: 串行 vs 并行

Karpathy 反复强调 RNN 虽然理论上 “可以 implement arbitrary programs”, 但它们 “are serial computing devices”, 图展开非常长, 无法高效并行。相比之下, Transformer 是 “shallow wide graph”, 从监督到 input hop 数少, 梯度也更容易穿过。

深度循环网络的警示

- RNN 训练时 hop 数众多, backpropagation step 也多, 容易出现梯度爆炸/消失
- 串行展开的 compute graph 限制了大批量并行训练
- 在工业级训练中, RNN 很难在 compute/d cost 竞争力上对抗 Transformer

3.5 本章小结

Karpathy 的 scaling 逻辑: 资源向 compute/data 倾斜、用 prompt 组织 “thick” 指令流、摒弃串行图, 换成 Transformer 的并行路径。这个节奏指导了后续所有训练策略。

4 应用与案例剖析

4.1 minGPT / nanoGPT 的教学案例

在 31 分钟左右, Karpathy 分享自己 “reproducing GPT-2 on open WebText” 的项目, 并把 minGPT/nanoGPT 作为 “快速上手” 的示范, 证明只要合理调参数, Transformer 微型版本也能复现核心行为。

小模型也能讲大道理

minGPT 通过在小型架构上训练 GPT-2 复制品, 让我们看到 Transformer 架构的可解释性: Attention map 可以可视化、layer norm 位置可以调整、每层 residual 贡献都可度量。

他强调 minGPT/nanoGPT 所用的数据集在苏醒的人力资源 (如 OpenWebText) 上构建, 允许学生重新运行训练流程, 从数据准备、超参 sweep 到生成样本, 都在单机 GPU 可控范围内完成, 降低学习门槛。

开放源码复现的意义

通过开放代码与权重，学生可以观察 attention heads、layer norm 影响，甚至在自己小型任务上做微调，从而把 “Transformer 的算法” 转化为 “可重写的工程”。

4.2 多模态与 RL 方向的示例

Karpathy 提到 Whisper 和 Decision Transformer: Whisper 是 “copy based Transformer”，将 spectrogram 当作 “text” 来处理；Decision Transformer 则利用扩展 prompt 载入 trajectory 片段，直接按 sequence 生成行为。这一思路展示了 Transformer 不再局限语言，而是跨模态使用相同计算图。

跨模态的 Transformer 处理模式

将非文本输入 (spectrogram、RL trajectory) 编码为 “token”，再交给 Transformer 处理；输出则用不同 head 解码 (音频 frame、动作)。因为 self-attention 只关注 token 的关系，跨模态扩展只需在 tokenizer/decoder 上做工程。

Karpathy 进一步提示，我们可以把 spectrogram 展开为序列，再让 self-attention 学习 frequency 之间的关系；对于 RLtrajectory，只需把 reward + state + action 串成 token，Transformer 就能像语言模型一样生成下一步动作。

4.3 未来趋势与人机交互

Karpathy 最后调侃自己 “电脑上 200 个 open tabs”，同时感叹 “we should be kind of writing years”——这是在表达人类需要向 “AI 辅助的天才交互” 进化，将 Transformer 的 prompt 设计作为新形态的编辑器。

他的论调把 Transformer 当作 “可编程界面”：我们不是在训练一个静态程序，而是在 prompt 中编排一系列动作，通过 attention 逐步渲染出用户需要的答案。这个趋势告诉我们，未来的工程师既要懂 Data 也要懂 prompt 组织。

4.4 Slides/Frames 与素材策略

本讲的封面已经采用课程官方的 cover slide (见第 1 节)，我们还可以通过额外截图或 slide PDF 拓展插图。若存在 slides 数据 (如 slide PDF 或 Google Slides)，应优先将截取后的 jpg 作为 figure；若只有视频，也需要抓取 key frame。当前目录只保存了 cover.jpg，因此后续如果下载到补充 slide，务必统一放在 ‘slides-images/’ 目录并引用。

素材类型	用法细节
Cover slide	直接作为封面 figure, 以 00:00:05–00:00:20 时间段说明来源
Key frame (若有)	以 ‘frame-XX.jpg’ 命名, 放入 ‘frames/’, 在 caption 中标注具体时间
Slides PDF	使用 ‘magick -density’ 提取主讲页面, 优先用于关键概念页

表 4: 用户素材 (slides/frames) 采集与使用策略摘要。

4.5 本章小结

minGPT/nanoGPT、Whisper、Decision Transformer 这些例子都让同一个 Transformer 架构在不同模式、不同数据结构上复用, 说明 “only the data structure around the tokens” 需要调整。

5 工程实践与复现

5.1 数据与计算预算

这场讲座提醒我们, scale first 不只是加大模型参数, 也需要把数据的质量和 compute budget 对齐。Karpathy 说 “worry about compute and data”, 因此我们要:

- 收集 OpenWebText、Common Crawl 等大规模语料, 并剔除重复与感官敏感内容
- 用 BPE/Tokenizer 将句子分段为 512 1024 tokens, batch 维度根据 GPU memory 调整
- 规划 compute budget: Token 数 $\times$ 模型参数数 $\times$ epoch 数给出实际 FLOPs

维度	建议	说明
数据	去噪 + dedup	保留 high-quality token, 避免训练时 repeat
Compute	Token budget	例如 1e12 1e13 token 数量确保泛化
Infrastructure	Mixed precision + gradient checkpoint	降低显存占用, 保持 throughput

表 5: Scale-first 模型的实践预算参考。

数据与 compute 的协同

在训练的早期阶段, 控制数据质量比盲目加模型更重要; 到了 later stage, 再用 larger compute 细调模型。这个过程就是 Karpathy 常说 “build big first, then polish with data” 的具体体现。

5.2 调试与观察

确保训练稳定, 需要实时监控 loss、attention maps、embedding scales 等。

关键监控指标

- LayerNorm / residual 的均值与方差是否飘移
- Attention 图中是否有某些 head 恒定注意同一个位置
- 学习率 warmup 曲线能否平滑过渡到 plateau

此外，可以生成小批次的 sample，如 prompt: “Translate English to German”，快速检验模型是否保留在 prompt 指定的任务上。

5.3 复现脚本示例

下面是一个简化的训练循环（以 minGPT 为蓝本），帮助工程师迅速落地：

```
1 from torch import nn, optim
2
3 model = SimpleTransformer(vocab_size=50257)
4 optimizer = optim.AdamW(model.parameters(), lr=6e-4)
5
6 for epoch in range(num_epochs):
7     for batch in dataloader:
8         logits = model(batch["input_ids"])
9         loss = nn.functional.cross_entropy(logits[:, :-1].reshape(-1, logits.size
            (-1)), batch["input_ids"][:, 1:].reshape(-1))
10        optimizer.zero_grad()
11        loss.backward()
12        optimizer.step()
```

Listing 1: 简化 minGPT 训练循环

实践中的注意事项

- 切勿在自动微分后忽略 `grad_clip`，否则较大的 token 时长容易爆炸
- 不了解模型 scale 时，尽量先在 1-2 张 GPU 上跑完一轮

5.4 本章小结

通过明确的数据/compute 预算、可解释的监控指标与示例训练代码，我们才能把 Karpathy 的 scaling 思维在工程中落地。

6 Transformer 演化与 Karpathy 视角

6.1 关键变体对照

Karpathy 在讲座中提到三个主流变体：BERT、GPT、T5(或 BART)等，它们沿用了 Transformer 的核心结构，但在 attention 方向、训练目标与位置编码上做出不同选择。

模型	训练目标	Karpathy 的观察
BERT	Masked LM, 聚焦双向理解	提供对 token 关系的深度表示, 但不适合自回归生成
GPT	自回归, 因果 attention	stack size 可延展, 适合 prompt / playback 交互
T5 / BART	Encoder-Decoder, seq2seq	兼顾翻译与摘要, 强调结构化输入

表 6: Karpathy 讲座中提到的 Transformer 变体与定位。

Karpathy 的稳定性观察

他指出 “Transformer architecture hasn’t changed in 5 years”, 说明 attention + residual 的基本组合已经足够强大, 其扩展更多是周边细节 (Position encoding、layer norm 的对称性)。

6.2 Karpathy 对稳定性的解读

尽管各个变体在 component 上有差异, 但 Karpathy 认为整个系统仍旧围绕 “self-attention + residual + MLP” 展开, 其他 90

稳定性与创新的平衡

模型架构的稳定性给了我们 “史诗级可复现” 的基础, 具体任务的改进则通过数据、prompt、RLHF 实现, 而不是不断堆叠新结构。

6.3 本章小结

Karpathy 对 Transformer 演化的观点强调核心结构的稳定性与周边优化的可插拔性, 这种视角帮助我们明白 “为什么重复使用同一个架构还会成功”。

7 讲座时间线与主题剖析

7.1 拆解主要章节

时间段	主题	亮点
00:00–00:06	课程定位	Karpathy 强调 Transformer 不是机器人，更像是通用计算器
00:06–00:18	架构解构	自注意力、LayerNorm、残差路径与 scaling 之间的关系
00:31–00:47	minGPT 实践	复制 GPT-2、论述开源模型对教育的价值
00:53–01:02	Whisper / Decision Transformer	Multi-modal + RL 的延伸
01:02–01:07	未来趋势	prompt 作为新编程界面，以及对 AI 助手的展望

表 7: Lecture 11 的时间线与主轴。

时间线的学习价值

按时间段拆解讲座有助于在复盘时快速定位核心内容，并决定哪些部分需要复看或加入笔记。

7.2 Q&A 与反思

Karpathy 在现场回答问题时说过：“RNNs are serial, Transformers are shallow wide graphs”，并坦言 “most progress comes from scaling compute/data, not architecture tweaks”。这种语境帮助我们在未来读文献时分辨 hype 与实干。

现场问答启发

- “RNN 能实现任意程序，但效率很低”——说明研究应该把精力放在并行路径
- “Prompt is like designing a thick sequence”——我们可以把 prompt 迭代看作人类编程 AI

7.3 本章小结

把讲座按时间段拆解，结合问答摘录，可以更系统地复现 Karpathy 的思维路径与重点句。

8 实践问题清单

8.1 常见问题与应对

在实践中，我们常问：数据集是否 clean？Compute cost 如何控制？Prompt 是否符合任务？下面整理一个清单：

1. 数据重叠率高？⇒ 引入 dedup 脚本 + 语义相似度过滤

2. Token 预算超支? $\Rightarrow$ 降低 seq_len 或 batch, 并开启 gradient checkpoint
3. Loss 曲线振荡? $\Rightarrow$ 检查 warmup, 尝试更小 learning rate 或 gradient clipping
4. Attention 预热异常? $\Rightarrow$ 可视化 attention map 看是否稀疏
5. RLHF 不稳定? $\Rightarrow$ 先锁定 reward model, 逐步放开 PPO
6. Prompt 表现不好? $\Rightarrow$ 加 few-shot 或 chain-of-thought 示例

问题	应对策略	工具提示
高 loss 波动	增加 grad_clip + warmup	TensorBoard / WandB 可视化
Prompt 无响应	加 role + examples	使用 prompt caching 表示
Compute 受限	Mixed precision + gradient checkpoint	Apex / torch.compile

表 8: 实践问题与常用工具。

8.2 工具与自动化

Karpathy 还建议把训练过程与 prompt 迭代写入文档, 配合 scripts 自动化 (如 ‘python train.py --config’)。工程师应在每次 run 后记录 ‘config’, ‘seed’, ‘throughput’, 确保后续复现。

可复现实践要素

把 config、seed、optimizer state 与 prompt design 放在 version control, 配合 summary table, 能显著提升跨团队协作时的信任度。

8.3 本章小结

有了实践清单与自动化工具, 我们就能把 ‘Karpathy 讲 high-level idea’ 变成 ‘工程师可执行的 pipeline’。

9 Prompt 协同与人类交互

9.1 Chain-of-thought 与 prompt 结构

Karpathy 强调 prompt 并不是被动输入, 而是 ‘thick sequence’ 的主动编排: 在 prompt 前端注入角色设定、few-shot 示例、chain-of-thought 乃至调度约束, Transformer 会把整个 prompt 视为同一张计算图的一部分, 从而直接在生成时遵循规则。

Prompt 组成	作用	变体尝试
角色/格式说明	预定义模型输出的语气、结构	“Assistant is a translator”
Few-shot 示例	提供输入-输出 pair, 引导 attention pattern	Chain-of-thought 示例
Constraint Tokens	强调必须遵守的 guardrails	“Do not provide medical advice”
Post-amble	指定总结、引用等输出形式	“Summarize in bullet points”

表 9: Karpathy 讲解中 prompt 的结构化模板。

Prompt 作为程序规格

把 prompt 想象成一个嵌入式 DSL: 前端的说明 + 示例是编译器传入的 spec, Transformer 在内部图上执行, 并将 spec 视为 “next token” 的约束。优秀的 prompt 能把 task 的复杂度转化为 token 之间的关系, 而非修改模型参数。

9.2 Prompt 流程治理与 review

在团队实践中, 应让 prompt 的迭代记录在 ‘prompt log’ 中, 类似代码评审: 每次改动都要说明目的、预期效果与可能风险 (如 hallucination)。Karpathy 提到 “most progress comes from scaling compute/data, not architecture tweaks”, 但在 prompt 层面仍需严谨的 review 机制, 尤其当 prompt 被多个 downstream agents 调用时。

Prompt 迭代的常见陷阱

- 过度依赖 “magic token”, 忽视 token 长度带来的 compute 成本
- 没有复现环境 (seed/config) 就修改 prompt, 导致不同 run 行为漂移
- 把 prompt 结构当作 black-box, 不记录版本, 很难 troubleshoot

9.3 本章小结

Prompt 不只是输入, 它是 human-in-the-loop 的 orchestrator; 结构化 prompt 与治理流程让 Transformer 输出更可靠, 也让 engineer 可以审计每次交互。

10 评估与治理实务

10.1 评估矩阵与 failure signature

扩展模型容错能力的第一步是构建清晰的评估矩阵: Karpathy 演讲中的 “worry about compute and data” 同样适用于 evaluation, 多个维度 (准确性、鲁棒性、hallucination rate) 要同时跟踪。

维度	监控指标	工具
准确性	Prompt 输出命中率、BLEU/ME-TEOR	eval harness + 自定义 dataset
鲁棒性	长 prompt / adversarial token 处理能力	adversarial suites
Hallucination	不可验证事实比例	human annotation / cross-check API
风控	敏感内容触发率	强化指令 + filter

表 10: Lecture 11 中 Karpathy 所强调的多个 evaluation axis。

多维评估的价值

一个维度的提升可能掩盖另一个维度的退化。持续对照多个指标，可以在 prompt 或数据改动后立刻察觉 unintended drift，而不是等到线上问题爆发。

10.2 对齐治理的实战

治理过程中要制定 “prompt change impact” 报告，记录改动、实验 log 与人工评估结论。这份文档相当于 Karpathy 所说的 “build big first, then polish with data” 在控制层面的延伸：先搭建模型，再用具体指标调整 behavior。

对齐治理三部曲

- 记录 baseline prompt 与 human feedback 分数
- 施加策略 token/chain-of-thought，然后重新评估
- 将最稳定的版本锁定，进行 A/B 评估，对照各个维度

10.3 本章小结

通过多维评估与治理文档，我们能在 scaling 的背景下保持模型行为可预测并减少人工干预频率。

11 部署与可观察性

11.1 部署准备 checklist

Karpathy 强调 “worry about compute” 不只是训练，还包括 deployment：在上线前务必确认 prompt、token 预算、配套 compute budget（如 max tokens per request）都明确。下面的 checklist 体现了他的 “具体工程” 逻辑：

项目	要点	状态
Prompt stability	不同输入变体下输出一致性	
Token budget	计算单次请求 FLOPs, 与 GPU budget 对齐	
Monitoring hooks	loss/latency/throughput 全链路钩子	
Fallback plan	超时/异常时降级到 API 或缓存	

表 11: 部署前必须完成的核查项。

部署阶段的低级错误

忽略警告的 monitoring hook 是常见失误: Karpathy 在讲座中也提到 “scale first” 意味着在 failure 发生前就知晓, 并在 prompt/compute 的层面提前设置 guardrail。

11.2 可观察性与 drift detection

部署后继续监控 attention scales、token distribution、response length 等指标, Karpathy 指出 “Transformer 是 shallow wide graph”, 因此任意一层的 scale drift 可能立刻放大。可以结合 ‘WandB’、‘TensorBoard’ 等工具, 把 attention map、loss log 与 prompt 版本关联, 快速查明 root cause。

可观察性搭建要点

- attention map heatmap: 寻找 head 过度聚焦的位置
- LayerNorm scale: 低频 drift 可能意味着 optimizer 失衡
- Response length / token distribution: 提示 prompt 尾端的 “constraint tokens” 是否生效

11.3 本章小结

部署阶段依然是 scaling 的关键节点: 有了 checklist 和可观察性指标, 我们才能对 prompt / compute drift 做到快速反应, 保持 Transformer 作为 “general purpose text computer” 的可控性。

12 复现与研究方向

12.1 数据与模型构建 pipeline

Karpathy 在讲座中多次提及 “build big first, then polish with data”, 因此复现 minGPT/-nanoGPT 的第一步就是把训练数据与模型架构写成 checklist: 先准备 OpenWebText/crawl 数据, 再定义 tokenization、batch size、head 数等模型配置, 最后搭建 logging + sample 生成。

阶段	关键操作	复现提示
数据	Clean + dedup + tokenizer	先确认 dataset 的 dedup rate, 再保存 parser config
模型	定义 layers/head/embedding	minGPT 提供默认 config, 可直接 adapt
训练	Warmup + lr decay + grad clip	关注 attention scale 与 loss curve

表 12: 复现 minGPT/nanoGPT 时的 pipeline 分段。

Pipeline 复现要诀

把 config 与 training log 视为实验的“代码”：每次改动都用 git commit 保存、上传 artifact，并附上 sample 生成，以便后续团队可以无缝接手。

12.2 推理效率与对齐探索

随着模型规模扩大，推理阶段的 token 预算与 prompt stability 变成 bottleneck。Karpathy 不断重申 “worry about compute”，所以在部署前应预测每次调用的 FLOPs 并与 GPU budget 对齐。此外，可以把 prompt + context 的 token 双向记录，推测是否需要 slot-filling 或 streaming 机制来保持 prompt 结构。

高效推理的实践思路

- 使用 sliding window 来支持超长 context 且不扩展 prompt size
- 在 prompt 前端插入 summary token，压缩 context 而不丢语义
- 通过 attention map 可视化确认新的 context token 是否被有效利用

12.3 本章小结

把复现 pipeline 与推理优化当成两个并行的工程线程，可以确保 Karpathy 的 scaling 思路在研究团体内部既能复现也能快速交付。

13 团队协同与知识管理

13.1 角色与流程分工

Karpathy 讲述 scaling 时常常提到 “every GPU hour matters”，这催生了清晰的分工：数据/compute 侧、模型/对齐侧、prompt/产品侧都要有明确负责人。一个典型团队包括 Data Engineer（负责 dedup 与 tokenizer）、Model Engineer（负责 attention map + training log）以及 Prompt Engineer（掌管 prompt 结构与 evaluation）。

角色	核心任务	输出 artifact
Data Engineer	清洗/规范化语料, 规划 token budget	Dedup log + tokenizer config
Model Engineer	训练监控、LayerNorm/attention 可视化	loss curve + attention heatmap
Prompt Engineer	组织 prompt structure、设计 guardrails	prompt log + rule-book

表 13: Scaling 团队的分工与输出。

协同沟通的节奏

每周轮流 review 命令行对应的 prompt log、monitoring curve 与 sample outputs; 把 Karpathy 口中的 “build big first” 当成 sprint 目标再配合 prompt/gov review, 能避免各自孤岛。

13.2 知识捕捉与分享

Karpathy 的讲座本身就是一次大型知识捕捉: 他通过 slide、debug log 与 sample 生成, 把 scaling 的经验分享给学生。我们在复现时也需要把每一个实验的 lesson (如 prompt 约束、RLHF 弱点) 写成小结, 并贴进 repo 中供未来团队查阅。

知识管理实践

- 用 ‘notes/’ 目录存储每次 prompt/tuning 的 summary, 配合 ‘git describe’ 追踪
- 定期把重要 quotes (如 “worry about compute and data”) 写入学习卡片, 交给团队新人 review
- 用 ‘slides-images/’ 归档截图, 并在 LaTeX 里引用复刻讲座视觉风格

13.3 本章小结

团队协作和知识管理让 scaling 经验跨人传承: 分工明确、artifacts 可追溯、lesson 有地方沉淀, 才能把 Karpathy 的实践理念变成可持续的项目文化。

14 总结与延伸

维度	核心要点
架构解构	Transformer 是图上的消息传递，相邻 token 通过 self-attention 形成动态连接；LayerNorm 保持信号稳定。
训练哲学	先 build big (compute + data)，再 tune prompt；RNN 的串行拓扑难以竞争。
应用扩展	minGPT/nanoGPT 示范微型模型可做解释，Whisper/Decision Transformer 则展示跨模态 prompt 续航。

表 14: Lecture 11 的三条主线复盘。

14.1 拓展阅读

- Vaswani et al., “Attention Is All You Need,” NeurIPS 2017
- Karpathy, “minGPT,” GitHub repo
- OpenAI, “Training language models to follow instructions with human feedback,” NeurIPS 2022
- OpenAI, “Whisper: Open-source speech recognition in multiple languages,” 2022
- Chen et al., “Decision Transformer: Reinforcement Learning via Sequence Modeling,” NeurIPS 2021